# Bypassing the HBM Wall: A Distributed Spatial Processing-Near-Memory Architecture using DUV ASICs and Deterministic Routing

Bowen Gu
July 30, 2026

## Abstract

### Background/Objective

Modern Mixture-of-Experts transformer inference and high-performance scientific computing have collided with the Memory Wall[1]. Packaging constraints on advanced interposers artificially cap both memory capacity and thermal dissipation, while conventional Von Neumann instruction-fetching overhead consumes enormous energy and time merely to orchestrate data movement rather than perform useful compute[6]. This work aims to bypass the HBM capacity and cost wall through a distributed Processing-Near-Memory architecture using commodity memory modules and mature-node ASICs.

### Methods

We propose a distributed spatial Processing-Near-Memory (PNM) architecture that replaces centralized High-Bandwidth Memory (HBM)[15] with commodity LPCAMM2 LPDDR6 modules[16,17], each paired with a mature-node 14 nm or 28 nm DUV MAC ASIC on a rigid X/Y motherboard grid. A deterministic single-spine tree routing fabric—built from stateless Hardware Flit Repeater wormhole switches, dimension-order on-board routing, and combinational Z-axis ingress nodes—eliminates runtime operating-system scheduling, cache coherency protocols, and multi-path routing ambiguity[13,14]. Routing is pure coordinate arithmetic: an $O(1)$ function of a flit's [Layer ID — Module ID] header, with worst-case latency bounded in closed form at compile time. By compiling high-level intermediate representations ahead-of-time onto a physical topology schema exported directly from hardware discovery, the system maps Mixture-of-Experts transformer inference, FP64 stencil computations, and functional data-based workflows directly onto silicon[3,4].

### Results

A 512-node reference chassis provides 64 TB of attached memory and 87 TB/s of aggregate local bandwidth at roughly $4 per gigabyte—two orders of magnitude below HBM monoliths per byte—while a roofline analysis shows the target workloads are firmly bandwidth-bound and therefore fully served by mature-node logic[5].

### Conclusions

The architecture yields a spatial computer that executes immutable dataflow graphs in hardware with full determinism, provable deadlock freedom, and strict functional correctness, at a fraction of the thermal envelope and economic cost of monolithic GPU accelerators.

**Keywords:** Processing-Near-Memory; Wormhole Switching; Deterministic Routing; Dimension-Order Routing; Spatial Computing; LPCAMM2; DUV ASIC; Mixture-of-Experts Transformers; Stencil Computation; Roofline Analysis

## Introduction

### 1.1 The economic and physical bottleneck

The prevailing paradigm for large-scale transformer training and inference relies on monolithic GPU accelerators tightly coupled to High-Bandwidth Memory (HBM) via advanced interposers[2,15]. This approach has become economically and physically untenable. A single NVIDIA H100-class GPU with 80 GB of HBM can cost upwards of thirty thousand dollars—roughly $375 per gigabyte of attached memory—yet the majority of its transistor budget and thermal headroom is spent not on computation, but on moving data across hierarchical memory caches and network interfaces[2]. For Mixture-of-Experts (MoE) transformers, where hundreds of billions of parameters remain idle during any given forward pass[3,4], purchasing monolithic GPUs solely for their aggregate memory capacity is fundamentally broken: it imposes massive capital expenditure, extreme power density, and an artificial capacity ceiling dictated by interposer reticle limits. The Results section quantifies the alternative: first-order cost modeling places the proposed PNM node at roughly $4 per gigabyte of attached memory, approximately two orders of magnitude cheaper per byte.

### 1.2 Hardware as a static graph

This paper advances an alternative philosophy inspired by Unix pipelines, operating-system microkernels, and pure functional programming[6,7,8]. We treat the physical motherboard not as a passive substrate for a Von Neumann processor, but as an active, immutable dataflow graph. Computation should be a pure mapping of inputs to outputs—flits stream through hardware channels as byte sequences, and each hardware component does exactly one thing with zero state manipulation. There are no runtime page tables, no dynamic branch prediction, and no global memory coherency protocols. By enforcing functional immutability at the silicon level, the architecture eliminates entire classes of side-effects, race conditions, and operating-system scheduling overhead that plague conventional accelerators. This purity is not merely aesthetic: it is what makes bit-exact replay after faults (Section 4.1) and compile-time latency bounds (Section 4.3) possible at all.

### 1.3 Manufacturing realities: DUV versus EUV

A common objection to custom accelerator silicon is the perceived necessity of bleeding-edge process nodes. This architecture rejects that premise. Memory bandwidth, not transistor switching speed, is the binding constraint for inference and stencil workloads. The roofline analysis in the Results section makes this concrete: at the reference node's machine balance of approximately 1.5 FLOP/byte, both stencil neighborhoods and MoE expert weight-streaming sit deep in the bandwidth-bound regime[5], so a 14 nm MAC array is never the limiting resource. A 14 nm or 28 nm Deep Ultraviolet (DUV) ASIC is therefore perfectly sufficient to saturate a local LPDDR6 bus[17], and mature nodes offer incomparable advantages: dramatically lower mask costs, higher yields, established supply chains, and the freedom to use large die areas for MAC arrays without the exponential cost curve of EUV lithography. By pairing cheap, commodity LPDDR6 memory with mature-node logic, the system achieves cost-per-parameter ratios orders of magnitude below HBM-based monoliths.

The architecture described in this paper is the convergence of several distinct intellectual traditions in computer science and hardware engineering.

### 1.4 The Unix philosophy

The Unix philosophy holds that programs should do one thing well, accept streams as input, and compose via pipes[7]. We extend this principle to silicon. Each Hardware Flit Repeater (HFR) does exactly one thing: it forwards a flit to the next node. The Ingress ASIC compares a header bitmask and gates matching flits onto the local network. The MAC ASIC executes a tensor kernel on data present in its local LPDDR6 bank. There are no general-purpose cores, no speculative execution, and no dynamic reconfiguration on the hot path. The flit stream is the silicon analog of the Unix text stream: a universal, byte-oriented interface that decouples producers from consumers.

### 1.5 Functional and declarative programming

Functional programming treats computation as the evaluation of mathematical functions with no side-effects and no mutable state[6]. Conventional hardware is ruthlessly imperative: mutable registers, speculative branch rollback, cache invalidation protocols, and operating-system interrupts violate functional purity at every level. Our architecture inverts this relationship. The hardware itself enforces functional immutability: flits are immutable once injected, HFRs are stateless functions of their input, and MAC ASICs produce deterministic outputs from static memory contents. There is no global mutable state anywhere in the system. The entire machine is a single, spatially distributed pure function—a physical instantiation of a dataflow graph—making it a natural substrate for first-order, statically allocated functional programs (Section 2.14).

### 1.6 Microkernel architecture

Microkernels advocate that privileged software should do as little as possible and isolate the rest[8]. seL4 has been mathematically proven to enforce its security and isolation properties[9]. We extend this principle from software to hardware. The Z-axis Ingress ASIC is a microkernel boundary implemented in combinational logic. It enforces a strict, non-bypassable physical separation between layers: traffic cannot reach a layer without passing through that layer's ingress gate, which evaluates a hardware bitmask rather than a software access-control list. There is no configuration register to misprogram, no firmware to exploit, and no driver to crash—the isolation is a physical property of the silicon.

### 1.7 Application-specific integrated circuits

General-purpose processors derive flexibility from massive instruction decoders, register files, branch predictors, and cache hierarchies. An ASIC burns transistors only on the logic that directly contributes to the target computation. Fixed-function tensor accelerators have demonstrated this principle at scale[10], but they still rely on advanced process nodes and monolithic fabrication. Our architecture applies the ASIC philosophy to mature DUV nodes and commodity memory: the DUV MAC ASIC contains no instruction decoder, no branch predictor, and no operating-system interface—only a fixed-function MAC array hard-wired to its local LPDDR6 bus.

### 1.8 Historical precedents: dataflow machines and the Transputer

The dataflow computing paradigm proposed machines where execution is triggered by data availability rather than a program counter[11]. The INMOS Transputer embedded a simple processor with point-to-point communication links on each chip, enabling fine-grained parallelism without shared memory[12]. Our architecture inherits the Transputer's philosophy of computing elements

with direct, deterministic communication, but replaces its general-purpose processors with fixed-function MAC ASICs and its serial links with high-bandwidth LPDDR6 buses. The wormhole routing fabric draws on Dally and Seitz's work on deadlock-free routing: deterministic routing is deadlock-free if and only if the channel dependency graph is acyclic[13,14]. Our fabric guarantees acyclicity by construction through monotonically ordered virtual-channel classes (Section 4.4).

## Methods

### 2.1 The Z-axis spine and ingress nodes

The network topology is a strictly deterministic single-spine tree—a spine-and-leaf fabric with exactly one spine. A central Z-axis spine runs vertically through the chassis, bridging parallel horizontal X/Y motherboards with high-density mezzanine connectors. Traffic between nodes on the same layer follows exactly one dimension-ordered X/Y path; traffic between layers additionally enters the spine at its own layer's attachment, travels monotonically along the spine to the destination layer's attachment, and continues across the destination board's dimension-ordered path. Because there is exactly one legal route between any pair of nodes, routing requires no search algorithm, no arbitration, and no adaptive logic—only coordinate arithmetic.

At each layer, a Z-axis Ingress ASIC evaluates the incoming flit header against a local hardware ID bitmask. Implemented as a combinational tree of XOR and AND gates on a 14 nm DUV node, this comparator operates in approximately 50–100 picoseconds without clocks, queues, or software intervention in the compare path. The comparator's match decision is registered on the local Network-on-Board (NoB) clock edge; clock-domain crossings use standard source-synchronous forwarding between HFRs. Matching flits are gated onto the local X/Y NoB; non-matching flits proceed along the spine. The ingress boundary acts as a rigid physical microkernel barrier[9].

The spine is the fabric's only bisection. It is provisioned as parallel high-speed lanes with an aggregate of at least 1 TB/s per direction in the reference chassis. The sizing rule is derived at compile time from the compiler's knowledge of every route. Intra-layer stencil halo traffic never enters the spine (Section 2.2), and MoE token payloads are kilobyte-class, so the worst-case cross-layer load remains modest. A fat-spine variant—adding lanes toward the root—scales this bound for larger stack heights.

### 2.2 The intra-layer fabric: dimension-order routing

Within a layer, HFRs form a direct X/Y grid that mirrors the motherboard's physical socket layout. On-board routes use X-then-Y dimension order: a flit travels along the X axis to the destination column, then along the Y axis to the destination socket[13,14]. Dimension-order routes are deterministic, minimal, and free of cyclic channel dependencies by the standard results. A halo exchange between neighboring grid cells is a single-hop route between adjacent sockets that never touches the ingress gate or the spine.

### 2.3 The motherboard and isothermal manifold

The horizontal plane consists of rigid X/Y motherboards populated with paired LPCAMM2 memory sockets and DUV MAC ASIC sockets (Section 2.4). Stacked vertically, these boards create a severe thermal challenge: LPDDR6 modules throttle above 85°C. At the reference operating point, each node dissipates approximately 15 W (about 5 W DRAM, 9 W MAC array, 1 W fabric share),

a fully populated 64-node board about 1 kW, and an eight-layer chassis about 8–10 kW including fabric and control losses. The chassis incorporates an Isothermal Parallel Manifold—rigid vertical fluid channels that inject identical-temperature coolant (30°C) simultaneously across all stacked layers. Balance is enforced, not assumed: each branch is fitted with a calibrated flow orifice sized during chassis assembly, holding each branch's coolant temperature rise within a $\pm 2$°C band. At approximately 15 L/min per chassis (water, 10°C coolant temperature rise), the worst-case DRAM junction temperature remains at or below 70°C—15°C below the throttling threshold.

### 2.4 LPCAMM2 integration, packaging, and sideband signaling

Each compute node centers on an LPCAMM2 memory module conforming to JEDEC JESD318[16], providing access to over 300 pins of LPDDR6 signaling[17]. The reference configuration assumes a **128-bit** memory interface (matching the current LPCAMM2 bus width), yielding $\approx$170 GB/s per node at 10.7 Gb/s/pin. JEDEC's LPDDR6 CAMM2 standard targets a wider **192-bit** bus (24-bit subchannel $\times$ 8), which would raise per-node bandwidth to $\approx$256 GB/s at the same pin rate; the architecture supports both widths with no change to the MAC ASIC or routing fabric. The DUV MAC ASIC sits adjacent to—rather than inside—the DRAM die, making this Processing-Near-Memory (PNM) rather than true Processing-in-Memory (PIM). Two packaging variants define adjacency:

Variant A — Socketed ASIC on the motherboard (reference design). Each LPCAMM2 socket is paired with an adjacent low-profile ASIC socket (land-grid-array or card-edge mezzanine). The ASIC is socketed rather than soldered: nodes can be populated, depopulated, and replaced in the field, matching the POST discovery sequence (Section 2.5) and the failure model (Section 4.1). The memory module remains an unmodified commodity part. Placing the two sockets within roughly 20 mm keeps the LPDDR6 interface short enough for commodity-grade signal integrity.

Variant B — Module-integrated ASIC (high-density option). The DUV MAC ASIC is mounted directly on the CAMM module substrate beside the DRAM packages. This yields the shortest memory interface and lowest interface power, at three costs: the module becomes a custom part, ASIC heat sits adjacent to the DRAM, and a failed ASIC retires the entire module.

Direct BGA attachment of the ASIC to the motherboard was considered and rejected: a permanent solder joint defeats field serviceability and contradicts the reconfigurable-inventory philosophy of Section 2.5. The reference configuration in the Results section assumes Variant A.

Because JESD318 fixes the memory connector pinout, three sideband signals travel on a separate low-pin-count board-level sideband header (in Variant B, a short flex tail from the module substrate):

DOORBELL_TRIG: asserted by the node's DMA engine only when (a) the received byte count equals the message-length field in the flit header and (b) the end-to-end message CRC validates. Because HFR forwarding along a single path is strictly in-order, byte-count equality detects complete delivery; a stream aborted mid-message can never satisfy both conditions. The full two-loop state machine is specified in Section 2.9.

NODE_ERR: a fatal fault line that bypasses the NoB entirely, streaming directly to the central controller (Section 4.1).

TOPOLOGY_RDY: asserted during Power-On Self-Test once voltage margins and link training

succeed.

### 2.5 Initialization and the POST Discovery Sequence

Initialization begins when the central microcontroller pulses sequenced voltage rails down the Z-axis spine. The controller emits a ping frame; each layer's Ingress ASIC propagates it along the local X/Y NoB. Every populated node that completes link training and Built-In Self-Test asserts TOPOLOGY_RDY. The controller aggregates responses into a live inventory of active Layer IDs and Module IDs.

This inventory is the Instruction Set Architecture for this boot instance. The controller exports a topology schema describing physical coordinates, per-link bandwidth, and deterministic latency between each pair. That latency is a closed-form function of the two coordinate pairs (hop count $\times$ per-hop delay), not a search result. Because the compiler backend targets this exact physical graph, the motherboard layout becomes the executable program structure. Reconfiguring the hardware—adding a board, removing a faulty node, or changing stack height—automatically generates a new ISA schema.

LPDDR6 is volatile, so static state must be loaded after discovery. The reference design supports streaming from the host over controller uplinks (Section 4.2), loading a fully populated 64 TB chassis in under ten minutes at uplink line rate, or optional per-node boot flash on the sideband header (roughly two minutes through 512 parallel streams). Weight loading is a one-time DMA phase completed before execution begins.

### 2.6 Software Stack and Execution Model

### 2.7 IR compilation

High-level user code—whether a MoE transformer, a weather simulation stencil kernel, or a functional dataflow graph—is first lowered into a standard Intermediate Representation (IR) such as MLIR[18]. The backend does not target CUDA, SIMD, or a conventional instruction set. It targets the physical topography schema exported during POST. Tensor operations, stencil neighborhoods, and functional dataflow nodes are mapped to specific LPCAMM2/DUV nodes as memory-resident weights and compute kernels.

### 2.8 Coordinate routing and dispatch

The compilation pipeline operates in two phases.

Ahead-of-Time (AOT) mapping. The compiler assigns weights, kernels, and grid subdomains to physical coordinates, then computes every route and latency in closed form: within a layer, X-then-Y between coordinates; across layers, a monotone spine traversal between layer attachments, bracketed by on-board paths. Both are $O(1)$ arithmetic functions of the [Layer ID — Module ID] pair—on a single-path tree there is exactly one route, so no pathfinding algorithm has anything to find.

Just-in-Time (JIT) dispatch. Dynamic tokens—such as MoE expert routing decisions—arrive at the central router. The router evaluates the same closed-form routing function, prepends a strict [Layer ID — Module ID] header, and injects. There is no runtime arbitration, no congestion sensing, and no adaptive routing. A modest 1 GHz dispatch stage issues one routing decision per clock ($10^9$ decisions per second); tokens are batched up to 128 per dispatch, placing sustained dispatch

capacity near $10^{11}$ tokens per second, orders of magnitude above the memory-bound execution rate. Optional per-layer dispatch replicas scale capacity linearly and remain deterministic because the routing function is pure.

### 2.9 The doorbell mechanism

A flit's lifecycle exemplifies zero-overhead activation. The JIT dispatcher injects the payload into the fabric. The combinational ingress comparator at the target layer gates the flit onto the X/Y NoB in sub-nanosecond time. Hardware Flit Repeaters pass the byte stream via DMA into the target node's local address space. The message header carries a length field; the node's DMA engine counts incoming bytes and computes the running message CRC. When the byte count equals the length field and the CRC validates, the DMA engine asserts DOORBELL_TRIG. This physical pin transition wakes the DUV MAC ASIC instantly; there is no software interrupt handler, no software polling loop, and no operating-system context switch.

Every node—whether a MAC node or a repeater serving a CAMM slot—runs the same hard-wired two-loop doorbell discipline. This is not software polling: no instructions are fetched and no program counter exists. The loops are the armed and drain states of a finite state machine burned into the doorbell logic:

WATCH (initial loop): continuously check the sentinel bit at the end address of the DMA target buffer. When the sentinel is set (byte count equals header length and CRC valid), interrupt WATCH and enter DISPATCH.

DISPATCH: fire the node's resident function. COMPUTE (PNM processor): execute the resident kernel, e.g., matrix multiplication over the landed payload and local weights. FORWARD (repeater at the CAMM slot): stream the payload onward to the next statically mapped node.

DRAIN (reverse loop): while results egress, check the sentinel; when transfer completes, clear the sentinel (bit now empty) and clear the initial payload data; restart WATCH.

The buffer is rigorously empty before the next message can arrive, making back-to-back activations safe without software synchronization. Because every route is a single path of strictly in-order HFR stages, byte-count equality is a sound completion test. An aborted stream always fails either the count or the CRC, so the doorbell never fires on a partial or corrupt payload.

### 2.10 Reliability and determinism

LPDDR6 provides on-die ECC; the MAC ASIC's memory controller adds conventional SECDED on the bus. Every message carries a link-local CRC per flit and an end-to-end CRC per message. Because every node is a pure function of its local memory contents and its input stream, re-injecting the same message reproduces the computation bit-exactly. Replay is safe by construction—there is no hidden state to diverge—which underpins both the fault-recovery protocol of Section 4.1 and bit-exact deterministic replay for debugging.

### 2.11 Target Workloads

### 2.12 Mixture-of-Experts transformers

Trillion-parameter MoE transformers present a capacity crisis for HBM-based systems: only a sparse subset of experts is active per token, yet the entire parameter set must be accessible[3,4]. Our architecture distributes expert weights across hundreds of cheap LPDDR6 pools. A one-trillion-

parameter model at INT8 occupies 1 TB—eight reference nodes—and at FP16, sixteen. A single 64 TB chassis holds tens of trillions of parameters with room for replication.

When a token requires a given expert, the central router evaluates the coordinate routing function, dispatches the token via wormhole routing to the holding node, and the doorbell fires. The local MAC executes and returns the updated hidden state. Three refinements matter in production: (i) hot-expert replication from offline activation histograms across $k$ nodes; (ii) expert capacity via elastic input buffers sized to at least one full message burst, with compiler-set capacity factors; (iii) dense components (attention projections, embeddings, output head) replicated on dedicated dense nodes per layer under the same doorbell discipline. Idle experts draw no dynamic compute power—only DRAM refresh current.

### 2.13 Scientific HPC stencil workflows

Climate modeling, computational fluid dynamics, and seismic imaging rely on regular FP64 stencil operations over massive grids. These workloads map onto the rigid X/Y grid of sockets: each grid subdomain becomes a physical node, and halo exchanges become single-hop dimension-order routes between neighboring sockets that never enter the spine. Because the stencil neighborhood is known at compile time, the AOT compiler generates exact routes with no runtime indirection, replacing cluster middleware such as MPI[19] with a coordinate function.

The roofline analysis of Section 3.2 shows why mature nodes suffice: a 7-point FP64 stencil has an arithmetic intensity of roughly 0.3–0.5 FLOP/byte, far below the reference node's machine balance of about 1.5 FLOP/byte[5]. Where numerics permit, mixed-precision halo storage raises effective intensity further. Deterministic worst-case halo-exchange latency is a compile-time constant (Section 4.4).

### 2.14 Functional data-based workflows

The mappable subset is first-order dataflow graphs of pure kernels over statically allocated buffers[6,11]. Within this subset, an actor is a node, a message is a flit, and a statically scheduled functional kernel is a doorbell activation. General lazy graph reduction with an unbounded dynamic heap is future work (the Conclusion section). Referential transparency is exactly what makes the replay guarantees of Sections 2.6 and 4.1 hold.

## Results

All figures in this section are first-order engineering estimates intended to establish plausibility and sizing rules, not measured results; no prototype exists yet.

### 3.1 Reference chassis

Table 1 summarizes the reference configuration: 512 nodes, 64 TB aggregate capacity, approximately 87 TB/s aggregate local bandwidth, approximately 131 TFLOPS FP64, and 8–10 kW chassis power, with a spine provisioned at $\geq$1 TB/s per direction.

### 3.2 Roofline: why mature nodes suffice

The reference node's machine balance is

Table 1: Table 1. Reference chassis configuration (first-order estimates).

| Quantity | Reference value |
|---|---|
| Node | 128 GB LPCAMM2 LPDDR6 + 14 nm DUV MAC ASIC (128 FP64 FMA @ 1 GHz ≈ 256 GFLOPS FP64) |
| Node memory bandwidth | ≈170 GB/s (128-bit LPCAMM2 @ 10.7 Gb/s/pin); up to ≈256 GB/s with JEDEC 192-bit LPDDR6 CAMM2 bus |
| Node power | ≈15 W |
| Board (1 layer) | 8×8 = 64 nodes; 8 TB; ≈10.9 TB/s; ≈1 kW |
| Chassis (8 layers) | 512 nodes; 64 TB; ≈87 TB/s; ≈131 TFLOPS FP64; ≈8–10 kW |
| Spine | Parallel lanes, ≥1 TB/s per direction aggregate |

$$B_{\text{machine}} = \frac{256\ \text{GFLOP/s}}{170\ \text{GB/s}} \approx 1.5\ \text{FLOP/byte}.$$

A 7-point FP64 stencil has arithmetic intensity ≈0.3–0.5 FLOP/byte (strongly bandwidth-bound). MoE expert weight-streaming sits near 1–2 FLOP/byte per token (higher with batching). Both sit at or below machine balance[5], so performance is limited by the LPDDR6 bus, never by the MAC array. This is the load-bearing argument for DUV: a 14 nm or 28 nm array already saturates the memory it is attached to.

### 3.3 Cost model

First-order, volume-dependent node bill of materials: LPCAMM2 module ≈$400, 14 nm MAC ASIC ≈$40, socket/board/assembly share ≈$60, for a node total of ≈$500 (≈$3.9/GB). A 512-node chassis is ≈$256k for 64 TB. An H100-class GPU at ≈$30k for 80 GB HBM is ≈$375/GB[2]. The architecture delivers roughly two orders of magnitude lower cost per gigabyte of attached memory.

### 3.4 Thermal and latency budgets

Thermal: ≈15 W per node, ≈1 kW per board, ≈8–10 kW per chassis; 15 L/min of 30°C water with 10°C rise and orifice-balanced branches keeps worst-case DRAM junction ≤70°C against an 85°C throttle.

Latency (bounded, not average): ingress compare ≈0.1 ns; HFR hop ≈1–2 ns; worst-case on-board route (8×8) 14 hops ≈14–28 ns; spine traversal (7 layer hops) ≈7–14 ns; 4 KB DMA @ 170 GB/s ≈24 ns; doorbell wake ≈1 ns; end-to-end activation bound ≈45–70 ns.

## Discussion

### 4.1 Failure Model and Recovery

NODE_ERR assertion triggers a four-step protocol: (1) quarantine the node from the live topology schema; (2) discard in-flight flits destined for it (the affected set is exactly enumerable on a single-path fabric); (3) replay—the source re-injects toward a healthy replica or the reinitialized node;

replay is bit-exact by the purity guarantee of Section 2.6; (4) re-export the amended schema and update JIT tables at the next batch boundary. The chassis continues in degraded mode with no restart. A failed spine lane degrades to remaining lanes; total spine failure partitions the chassis (the Conclusion section).

### 4.2 Host Interface and I/O

The central controller terminates dual PCIe Gen5 x16-class uplinks (≈128 GB/s aggregate) or 100/200 GbE equivalents. The host streams initial weights at boot, injects input tokens, and drains outputs. Runtime token traffic is kilobyte-class at the dispatch rates of Section 2.6—well below uplink and spine capacity. The host never touches the compute fabric directly.

### 4.3 Verification and Formal Properties

Deadlock freedom (sketch). Every flit is assigned one of three virtual-channel classes in strictly increasing order: (0) on-board egress (X-then-Y dimension order), (1) spine traversal (monotone along the linear spine; upward and downward flows on separate virtual channels), (2) on-board delivery (X-then-Y from spine attachment to destination). Intra-class dependencies are acyclic by standard results; cross-class dependencies only point from lower to higher classes. The global channel dependency graph is therefore acyclic, and by Dally and Seitz the fabric is deadlock-free[13,14].

Livelock freedom follows from minimal single-path routes. Consumption is enforced structurally: elastic input buffers hold at least one maximum-length message, and sources may not inject without destination buffer credit. Worst-case latency for any coordinate pair is a closed-form expression (Section 3.4). Given identical inputs, every computation and message schedule reproduces bit-exactly.

### 4.4 Limitations and Future Work

LPDDR6 (JESD209-6)[17] parts and high-capacity LPCAMM2 modules are forward-looking; an LPDDR5X variant (≈136 GB/s per node, 48–64 GB modules today) trims bandwidth by about 20 percent and capacity by half without architectural change[16,17]. The reference design assumes a 128-bit LPCAMM2 bus (≈170 GB/s per node), but JEDEC's LPDDR6 CAMM2 standard targets a 192-bit bus, which would raise per-node bandwidth to ≈256 GB/s; the architecture accommodates either width. Manifold balancing requires CFD and hardware validation. Multi-chassis scale-out needs a second routing level and one additional virtual-channel class. General lazy evaluation needs a dynamic heap this design omits. A dual-spine variant eliminates single-chassis partition mode. The claims of Section 4.3 are argued, not yet machine-checked[9].

Conclusion

This paper has specified a distributed spatial Processing-Near-Memory architecture that bypasses the HBM capacity wall through commodity LPDDR6, mature-node DUV logic, and deterministic wormhole routing[13,15,17]. By treating the physical motherboard as an immutable dataflow graph and exporting its topography as a bespoke ISA, the system eliminates runtime scheduling, cache coherency, and operating-system overhead. Routing is coordinate arithmetic on a single-spine tree with dimension-order on-board paths; deadlock freedom follows from monotonically ordered virtual-channel classes; the hardened doorbell mechanism makes activation atomic as well as idle-free; and the isothermal parallel manifold sustains thermal viability with enforced flow

balance. The reference chassis—64 TB, 87 TB/s aggregate local bandwidth, approximately 131 TFLOPS FP64, approximately $4/GB—demonstrates that the economics of memory capacity favor radical simplicity by roughly two orders of magnitude per byte. Target workloads—MoE transformer inference, FP64 stencil computation, and functional data-based evaluation—sit in the bandwidth-bound regime where mature-node silicon is already sufficient[3,4,5], and they inherit bit-exact replay and compile-time latency bounds that no cache-coherent machine can offer.

All hardware schematics, topology specifications, and PCB layouts described herein are intended for release under the CERN Open Hardware Licence Version 2 Strongly Reciprocal (CERN-OHL-S)[20].


## Acknowledgments

The author thanks their research mentor for guidance throughout this project, the anonymous referees for their constructive feedback, and the open-source hardware community for developing the tools and ethos that made this work possible.


## References


1. W. A. Wulf, S. A. McKee. Hitting the memory wall: implications of the obvious. *ACM SIGARCH Computer Architecture News*. Vol. 23, pg. 20-24, 1995, https://doi.org/10.1145/216585.216588.

2. J. Choquette. Nvidia hopper h100 gpu: scaling performance. *IEEE Micro*. Vol. 43, pg. 9-17, 2023, https://doi.org/10.1109/MM.2023.3256796.

3. N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, J. Dean. Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *arXiv preprint* arXiv:1701.06538, 2017, https://doi.org/10.48550/arXiv.1701.06538.

4. W. Fedus, B. Zoph, N. Shazeer. Switch transformers: scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*. Vol. 23, pg. 1-39, 2022.

5. S. Williams, A. Waterman, D. Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*. Vol. 52, pg. 65-76, 2009, https://doi.org/10.1145/1498765.1498785.

6. J. Backus. Can programming be liberated from the von Neumann style? a functional style and its algebra of programs. *Communications of the ACM*. Vol. 21, pg. 613-641, 1978, https://doi.org/10.1145/359576.359579.

7. M. D. McIlroy, E. N. Pinson, B. A. Tague. Unix time-sharing system: forward. *Bell System Technical Journal*. Vol. 57, pg. 1899-1904, 1978.

8. J. Liedtke. On micro-kernel construction. Proceedings of the fifteenth ACM symposium on Operating systems principles. pg. 237-250, 1995, https://doi.org/10.1145/224056.224075.

9. G. Klein, K. Elphinstone, G. Heiser, J. Andronick, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, S. Winwood. seL4: formal verification of an OS kernel. Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles. pg. 207-220, 2009, https://doi.org/10.1145/1629575.1629596.

10. N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snelham, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, D. H. Yoon. In-datacenter performance analysis of a tensor processing unit. Proceedings of the 44th annual international symposium on computer architecture. pg. 1-12, 2017, https://doi.org/10.1145/3079856.3080246.

11. J. B. Dennis. First version of a data flow procedure language. Programming Symposium. pg. 362-376, 1974, https://doi.org/10.1007/3-540-06859-7_145.

12. I. Barron, P. Cavill, D. May, P. Wilson. Transputer does 5 or more MIPS even when not used in parallel. *Electronics*. Vol. 56, pg. 109-115, 1983.

13. W. J. Dally, C. L. Seitz. Deadlock-free message routing in multiprocessor interconnection networks. *IEEE Transactions on Computers*. Vol. C-36, pg. 547-553, 1987, https://doi.org/10.1109/TC.1987.1676939.

14. W. J. Dally, C. L. Seitz. The torus routing chip. *Distributed Computing*. Vol. 1, pg. 187-196, 1986, https://doi.org/10.1007/BF01660031.

15. JEDEC Solid State Technology Association. High bandwidth memory (HBM) DRAM standard. JESD235D, 2021, https://www.jedec.org/standards-documents/docs/jesd235d.

16. JEDEC Solid State Technology Association. Compression attached memory module (CAMM2) common standard. JESD318, 2023, https://www.jedec.org/standards-documents/docs/jesd318.

17. JEDEC Solid State Technology Association. Low power double data rate 6 (LPDDR6) SDRAM standard. JESD209-6, 2025, https://www.jedec.org/standards-documents/docs/jesd209-6.

18. C. Lattner, M. Amini, U. Bondhugula, A. Cohen, A. Davis, J. Pienaar, R. Riddle, T. Shpeisman, N. Vasilache, O. Zinenko. MLIR: scaling compiler infrastructure for domain specific computation. 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO). pg. 2-14, 2021, https://doi.org/10.1109/CGO51591.2021.9370308.

19. Message Passing Interface Forum. MPI: a message-passing interface standard, version 4.0. 2021, https://www.mpi-forum.org/docs/mpi-4.0/mpi40-report.pdf.

20. CERN. CERN open hardware licence version 2 - strongly reciprocal (CERN-OHL-S). 2020, https://ohwr.org/project/cernohl/wikis/Documents/CERN-OHL-version-2.